COLLEGE LOGO

AN INTERNSHIP REPORT

ON

E-H0MES - BUY,RENT OR SELL YOUR HOME

Submitted by

STUDENT NAME(E_NO)

Student of

Diplomain Engineering

in

Your Department Name

COLLEGE LOGO

CONFIRMATION LETTER

COLLEGE LOGO

CERTIFICATE OF COMPLETION

COMPANY PROFILE

Established in 2016, incorporation with our parent IT company, INFOLABZ IT SERVICES PVT. LTD. has managed to make it's own position in IT Sector. We are involved in Web Development, App Development, Progressive Web Application Development, IOT solutions, Graphics & Designing, Digital Marketing, Domain & Hosting services, SMS services etc.

In the span of seven years we have managed to deliver all projects on time with utmost accuracy to our clients across the globe. We have dedicated teams of experienced andhard working developers. Our developers who are always willing to take new challenges and looking forward to learn new things, are heart of this company.

Our objective is to sustain with exponential growth in IT industry. Our mission is to deliver the best with top notch quality every quarter and vision is to develop a product with one of its kind concept which could be used by millions of people.

TABLE OF CONTENT

WEEK 1	**03 JULY 2024**	
	- BASIC MODELS CREATION	
WEEK 1	**04 JULY 2024**	
	- PROJECT WITH MULTIPLE TABLES AND DIFFERENT FORM FIELDS ADVANCED MODELS.	
WEEK 2	**05JULY 2024**	
	- CONCEPT OF FOREIGN KEY	
WEEK 2	**08JULY 2024**	
	- MATPLOTLIB	
	- GRAPHS : PIE CHART , LINE CHART, BAR CHART	
WEEK 2	**09JULY 2024**	
	- PROJECT WORK : DICTIONARY QUESTIONS, API ASSIGNMENT AND GRAPH QUESTIONS, DATA DICTIONARY	
	- INDUSTRY ASSIGNED PROJECT DEVELOPMENT	
WEEK 2	**10JULY 2024**	
	- ADMIN SITE UI MODIFICATION	
	- PROJECT REPORT EXPLANATION	

WEEK 1 :

PYTHON :

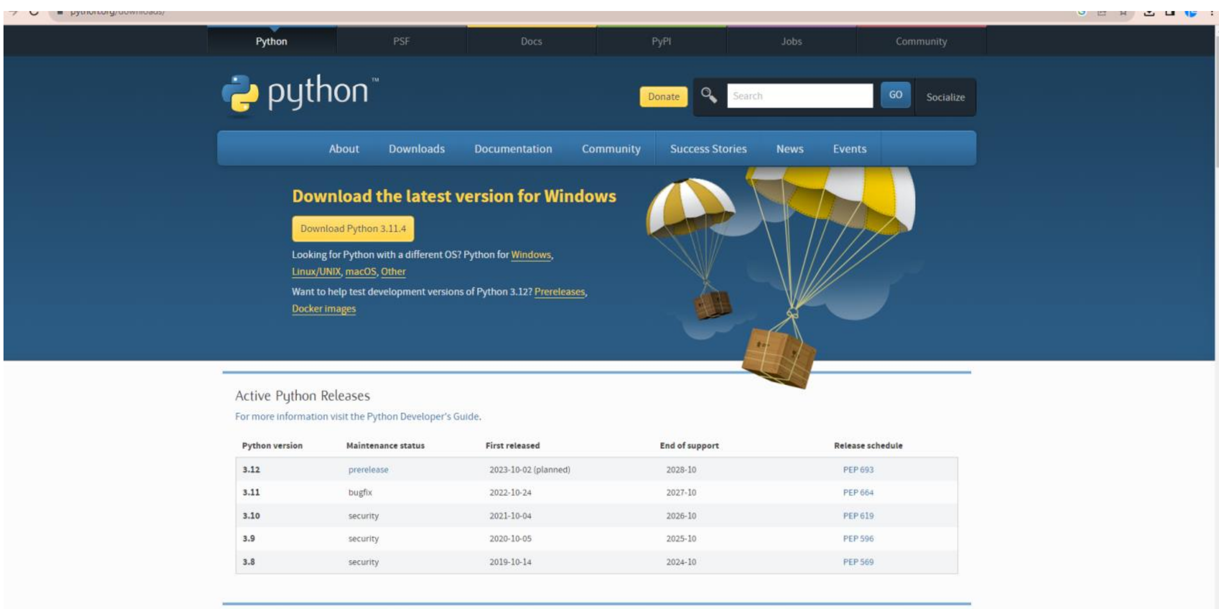
Python, created in 1991, is a popular, interpreted language with a clean syntax and strong community support. It offers dynamic typing and a rich standard library, making it suitable for a wide range of applications, from web development to scientific computing and machine learning. Its readability and ease of learning make it an excellent choice for programmers of all levels.

TOOLS:

PYTHON (INTERPRETER) :

URL : https://www.python.org/downloads/

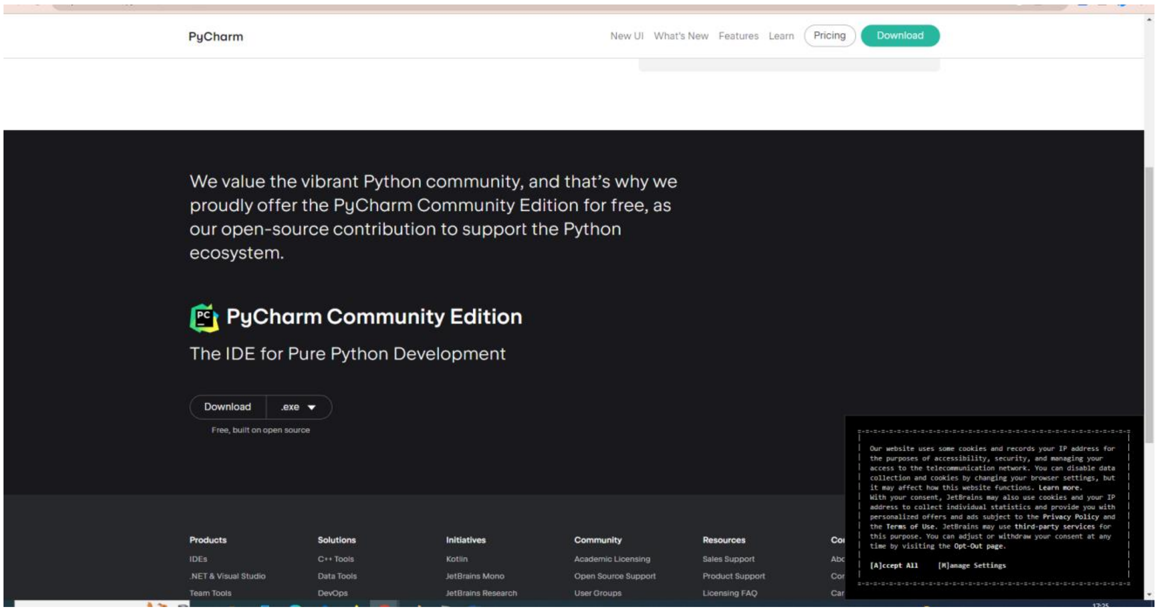
SIZE : 24.2 MB



PYCHARM (IDE) :

URL : https://www.jetbrains.com/pycharm/download/?section=windows

SIZE : 416 MB



PYTHON DATA STRUCTURE: Dictionary

A dictionary, often referred to as a "dict" in Python, is a built-in data structure used to store a collection of key-value pairs. Each key in a dictionary is unique and serves as an identifier that maps to a specific value. In Python, dictionaries are defined using curly braces {} and key-value pairs are separated by colons.

PROJECT 2 :E-H0MES - BUY, RENT OR SELL YOUR HOME

```
mydata = {
   "property_details": {
       "name": "abhishek appartments",
       "price": "1.40 CR",
       "is_available": True,
       "location":"vastrapur",
       "details": [
           {"type":"kitchen","area":210},
           {"type":"master room","area":290},
           {"type":"kids room","area":220},
           {"type":"guest room","area":210},
           {"type":"living room","area":310},
       ]
       },
  }
```

QUESTION 1 : print total number of Keys

QUESTION 2: print names of main keys

QUESTION 3:print name of apartment (abhishek appartments)

QUESTION 4:print living room area (310)

PROGRAM AND OUTPUT:

```python
    }

# print total number of main keys
print("total number of keys : ",len(mydata))
# print names of main keys
print(mydata.keys())
# print name of appartment ( abhishek appartments )
print(mydata["property_details"]["name"])
# print living room area ( 310 )
print(mydata["property_details"]["details"][4]["area"])
```

Run DICTIONARY2

```
C:\Users\Lenovo\AppData\Local\Programs\Python\Python311\python.exe C:\Users\Lenovo\Downloads\DICTIONARY_CODES\DICTIONARY_CODES\DICTIONARY2.py
total number of keys :  1
dict_keys(['property_details'])
abhishek appartments
310

Process finished with exit code 0
```

WHAT IS REQUEST PACKAGE?

The "requests" package in Python refers to the popular third-party library known as "Requests." It is a library designed to simplify the process of making HTTP requests, interacting with web services, and handling HTTP responses. This library provides a more user-friendly and higher-level interface for working with HTTP compared to using the built-in urllib module.

INSTALL PACKAGE:

Go to terminal -> pip **install requests**

WHAT IS API?

An API, or Application Programming Interface, is a set of rules, protocols, and tools that allows different software applications to communicate and interact with each other. It defines how different software components should interact, making it easier for developers to build applications that can use the functionality provided by other services or applications without needing to understand the internal details of those services.

API LINK 1: https://quote-garden.onrender.com/api/v3/quotes

QUESTION 1 : print name main of Keys

QUESTION 2 : print total number of main keys

QUESTION 3 : print the quoteAuthor : Hosea Ballou

PROGRAM AND OUTPUT:

```
import requests
response3=requests.get("https://quote-garden.onrender.com/api/v3/quotes")
data3 = response3.json()
print(data3.keys())
print(len(data3.keys()))
print(data3["data"][0]["quoteAuthor"])
```

```
C:\Users\Lenovo\AppData\Local\Programs\Python\Python311\python.exe "C:\Users\Lenovo\Downloads\DICTIONARY_CODES\DICTIONARY_CODES\dict api solution.py"
dict_keys(['statusCode', 'message', 'pagination', 'totalQuotes', 'data'])
5
Hosea Ballou
```

API LINK 2: https://api.spacexdata.com/v5/launches/latest

QUESTION 1 : print name of main Keys

QUESTION 2 : print total number of main keys

QUESTION 3 : print the rocket id(5e9d0d95eda69973a809d1ec)

PROGRAM AND OUTPUT:

```
import requests
response4=requests.get("https://api.spacexdata.com/v5/launches/latest")
data4 = response4.json()
print(data4.keys())
print(len(data4.keys()))
print(data4["rocket"])
```

```
C:\Users\Lenovo\AppData\Local\Programs\Python\Python311\python.exe "C:\Users\Lenovo\Downloads\DICTIONARY_CODES\DICTIONARY_CODES\dict api solution.py"
dict_keys(['fairings', 'links', 'static_fire_date_utc', 'static_fire_date_unix', 'net', 'window', 'rocket', 'success', 'failures', 'details', 'crew',
27
5e9d0d95eda69973a809d1ec
```

DATA VISUALIZATION :

Data visualization in Python involves crafting visual displays like charts, graphs, and plots to visually convey patterns and insights discovered within datasets. Python offers libraries like Matplotlib, Seaborn, Plotly, and more, empowering users to generate diverse visualizations. These visual representations aid in comprehending intricate data, recognizing trends, guiding decision-making, and simplifying the communication of findings to a broader audience

MATPLOTLIB :

Matplotlib stands as a widely adopted Python library designed to craft static, interactive, and animated visualizations, encompassing an array of charts, graphs, and plots. This library presents a versatile and adaptable framework that empowers the creation of diverse visual data representations. Renowned in data analysis, scientific exploration, and domains relying on data visualization, Matplotlib extends an extensive selection of plotting choices—ranging from line plots, bar plots, and scatter plots to histograms—rendering it a versatile instrument for articulating discoveries drawn from datasets.

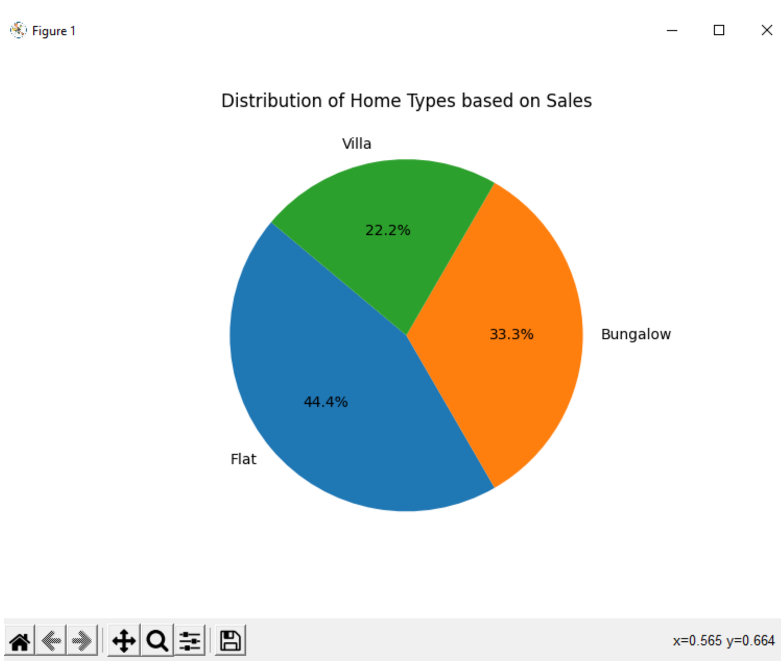
GENERATE GRAPH AS SHOWN IN OUTPUT FROM BELOW DATA

```
home_types = ['Flat', 'Bungalow', 'Villa']
sales = [40, 30, 20]
```

PROGRAM:

```python
import matplotlib.pyplot as plt

# Sample data (replace with your actual data)
home_types = ['Flat', 'Bungalow', 'Villa']
sales = [40, 30, 20]

# Create a pie chart
plt.pie(sales, labels=home_types, autopct='%1.1f%%', startangle=140)

# Adding title
plt.title('Distribution of Home Types based on Sales')


# Display the plot
plt.show()
```

OUTPUT::



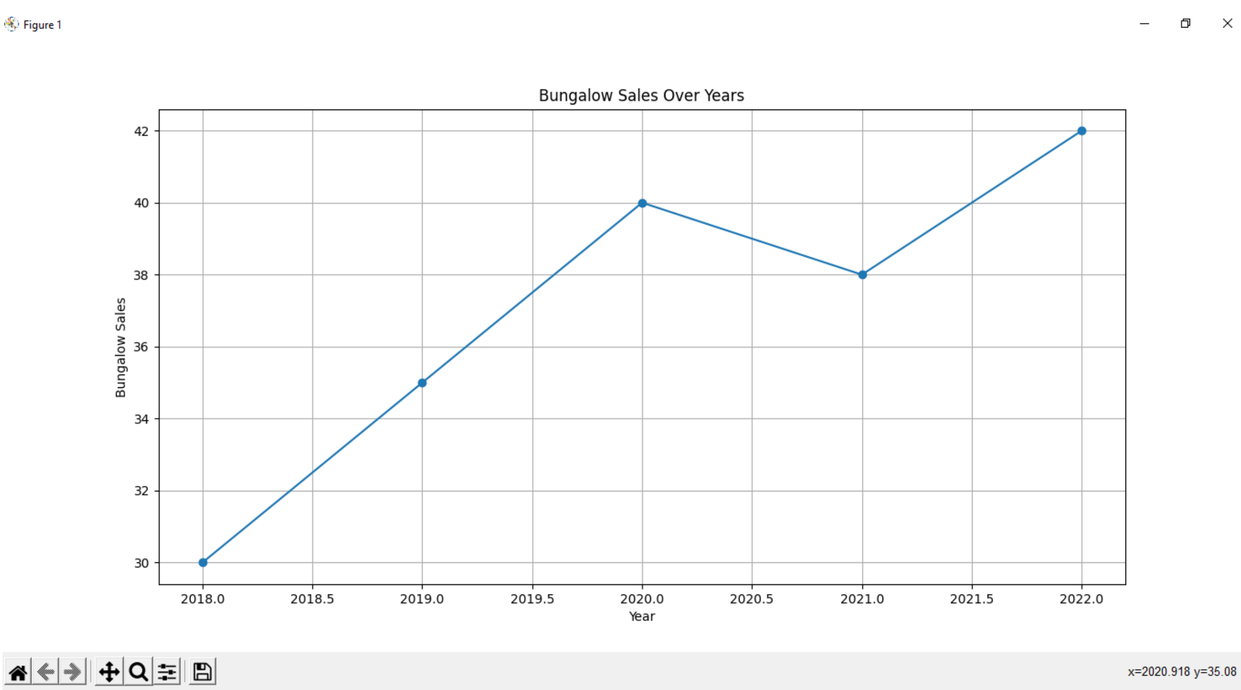
GENERATE GRAPH AS SHOWN IN OUTPUT FROM BELOW DATA

```
years = [2018, 2019, 2020, 2021, 2022]
bungalow_sales = [30, 35, 40, 38, 42]
```

PROGRAM:

```
import matplotlib.pyplot as plt

# Sample data (replace with your actual data)
years = [2018, 2019, 2020, 2021, 2022]
bungalow_sales = [30, 35, 40, 38, 42]

# Create a line graph
plt.plot(years, bungalow_sales, marker='o')

# Adding labels and title
plt.xlabel('Year')
plt.ylabel('Bungalow Sales')
plt.title('Bungalow Sales Over Years')

# Display the plot
plt.grid(True)
plt.show()
```

OUTPUT:



INTRODUCTION TO DJANGO FRAMEWORK:

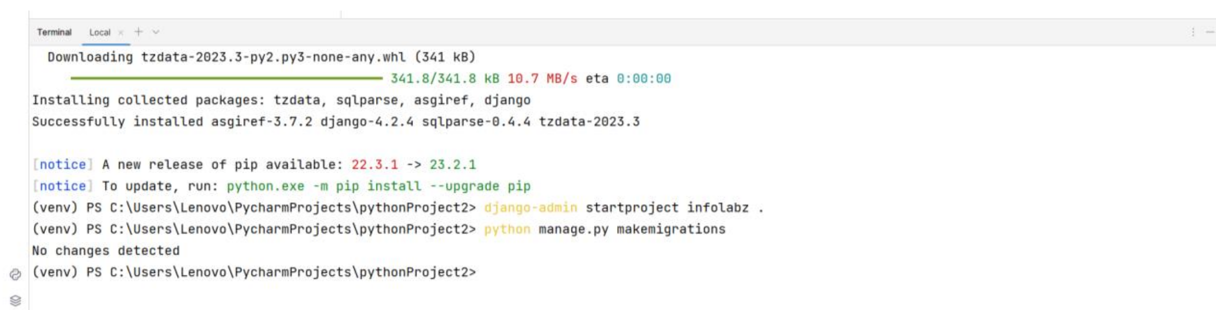
Django is a high-level, open-source web framework written in Python that enables developers to build robust, secure, and maintainable web applications rapidly. It follows the Model-View-Controller (MVC) architectural pattern, though in Django's case, it's often referred to as Model-View-Template (MVT). This framework is designed to simplify the process of creating complex web applications by providing a set of pre-built components and conventions that handle common tasks, allowing developers to focus on building unique features rather than reinventing the wheel.

STEP BY STEP DJANGO FRAMEWORK SETUP:

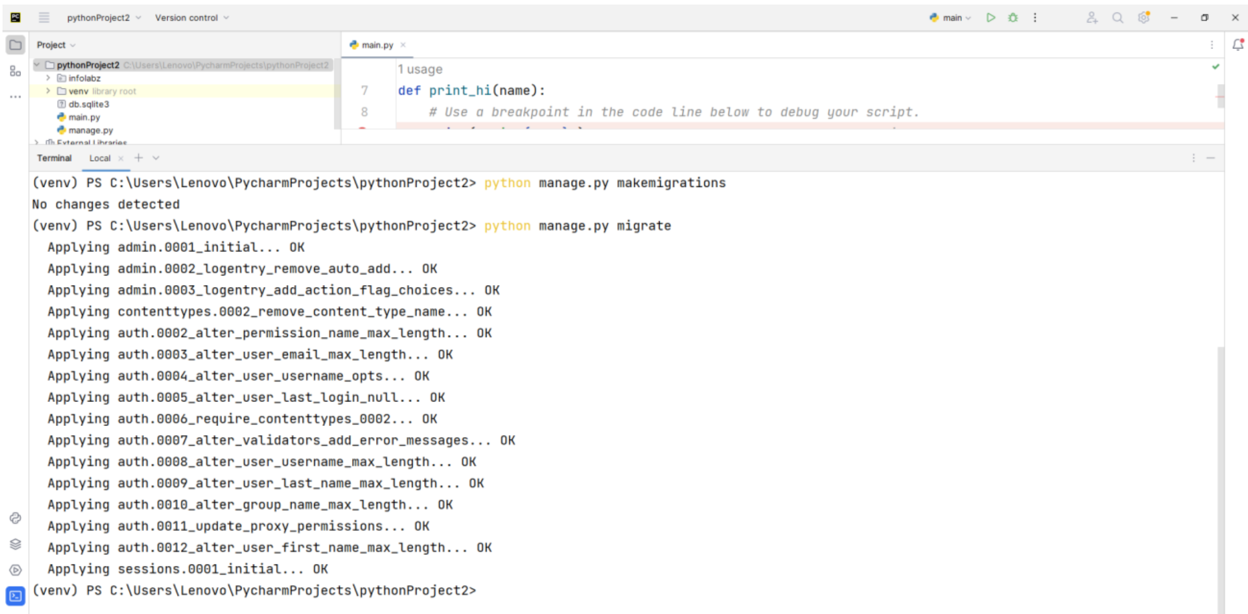
STEP 1 : Install Django Framework : pip install django

STEP 2 : Create Django project : django-admin startproject (project name) .

STEP 3 : Check for changes in db structure : python manage.py makemigrations

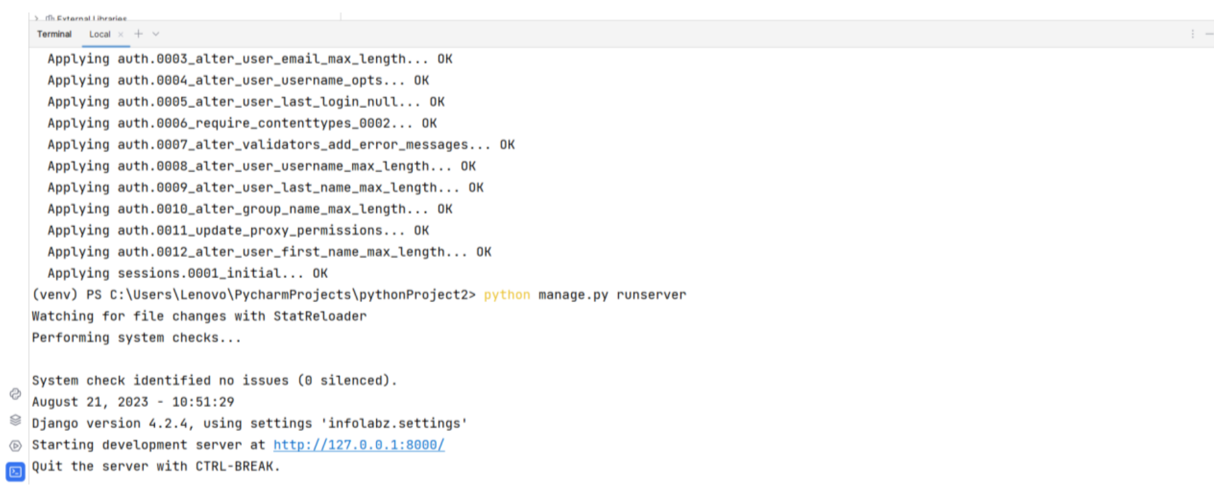


STEP 4 : Add / Migrate table in db : python manage.py migrate

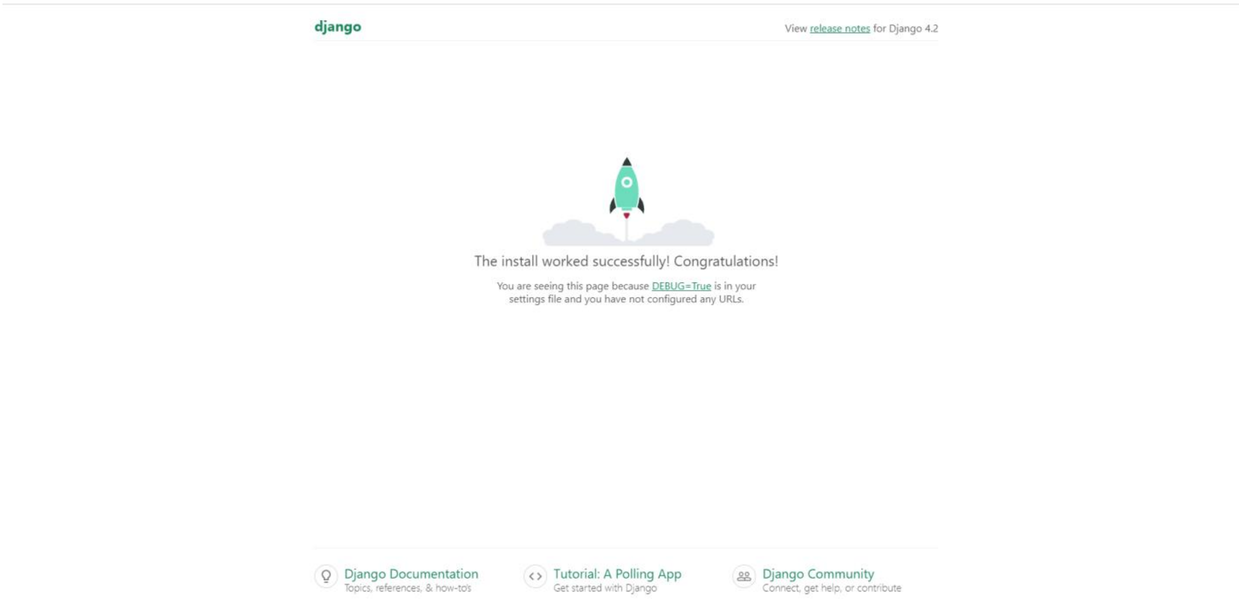


```
(venv) PS C:\Users\Lenovo\PycharmProjects\pythonProject2> python manage.py makemigrations
No changes detected
(venv) PS C:\Users\Lenovo\PycharmProjects\pythonProject2> python manage.py migrate
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
(venv) PS C:\Users\Lenovo\PycharmProjects\pythonProject2>
```

STEP 5 : Run project : python manage.py runserver

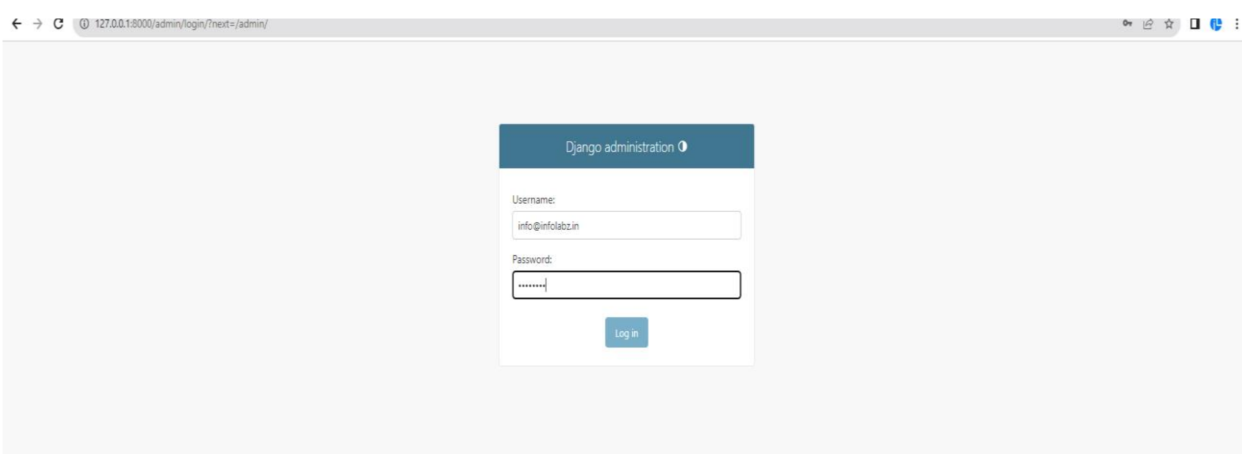


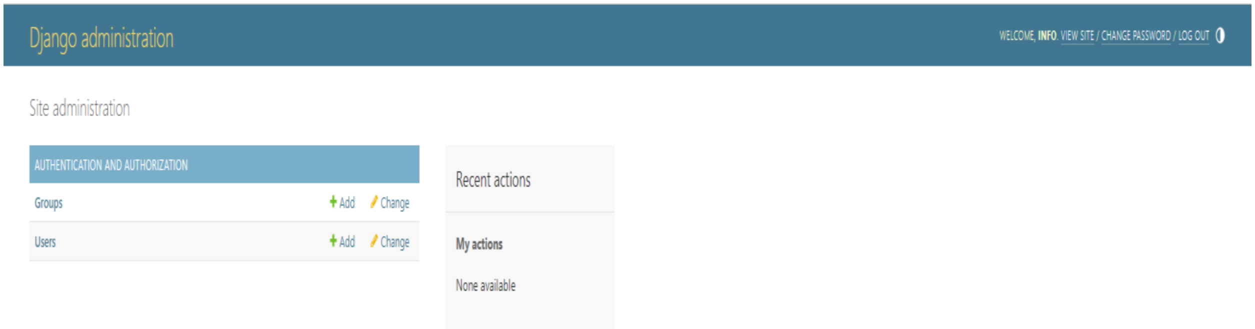
```
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
(venv) PS C:\Users\Lenovo\PycharmProjects\pythonProject2> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
August 21, 2023 - 10:51:29
Django version 4.2.4, using settings 'infolabz.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```



STEP 6: Create superuser : python manage.py createsuperuser

```
System check identified no issues (0 silenced).
August 21, 2023 - 10:51:29
Django version 4.2.4, using settings 'infolabz.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

[21/Aug/2023 10:51:50] "GET / HTTP/1.1" 200 10664
Not Found: /favicon.ico
[21/Aug/2023 10:51:50] "GET /favicon.ico HTTP/1.1" 404 2112
(venv) PS C:\Users\Lenovo\PycharmProjects\pythonProject2> python manage.py createsuperuser
Username (leave blank to use 'lenovo'): info
Email address: info@infolabz.in
Password:
Password (again):
Superuser created successfully.
(venv) PS C:\Users\Lenovo\PycharmProjects\pythonProject2>
```





PROJECT DATA DICTIONARY:

Table: User

Field Name	Field Type
Email	
Password	
Phone	
First Name	
Last Name	

Table: PropertyCategory

Field Name	Field Type
CATEGORY_NAME	Like flat , tenements

Table: City

Field Name	Field Type
CITY_NAME	

Table: Property

Field Name	Field Type
PROPERTY_NAME	
PROPERTY_DESCRIPTION	
PROPERTY_CATEGORY (Foreign key)	Referencing to PropertyCategory
LOCATION	
SELLER (Foreign key)	Referencing to User
PRICE	
BEDROOMS	
CITYID (Foreign key)	Referencing to City

Table: RentedProperty

Field Name	Field Type
PROPERTY (Foreign key)	Referencing to **PropertyCategory**
USER (Foreign key)	Referencing to **User**
START_DATE	
END_DATE	
TRANSACTION_DATE	
AMOUNT	
STATUS	DROPDOWN 1:CONFIRMED , 2-CANCELLED

Table: SoldProperty

Field Name	Field Type
PROPERTY (Foreign key)	Referencing to **PropertyCategory**
USER (Foreign key)	Referencing to **User**
TRANSACTION_DATE	
AMOUNT	

Table: Reviews

Field Name	Field Type
PROPERTY (Foreign key)	Referencing to **PropertyCategory**
USER (Foreign key)	Referencing to **User**
RATING	
COMMENT	
DATE_POSTED	

Table: FavoriteProperty

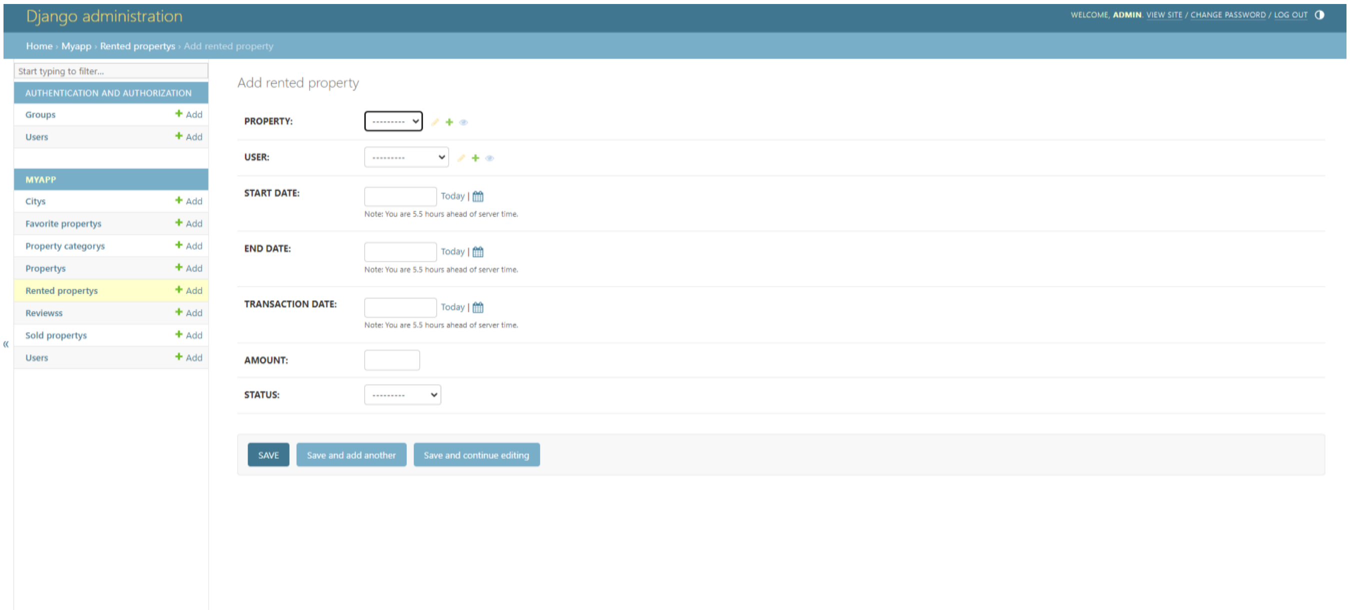
Field Name	Field Type
USER (Foreign key)	Referencing to **User**
PROPERTY (Foreign key)	Referencing to **PropertyCategory**

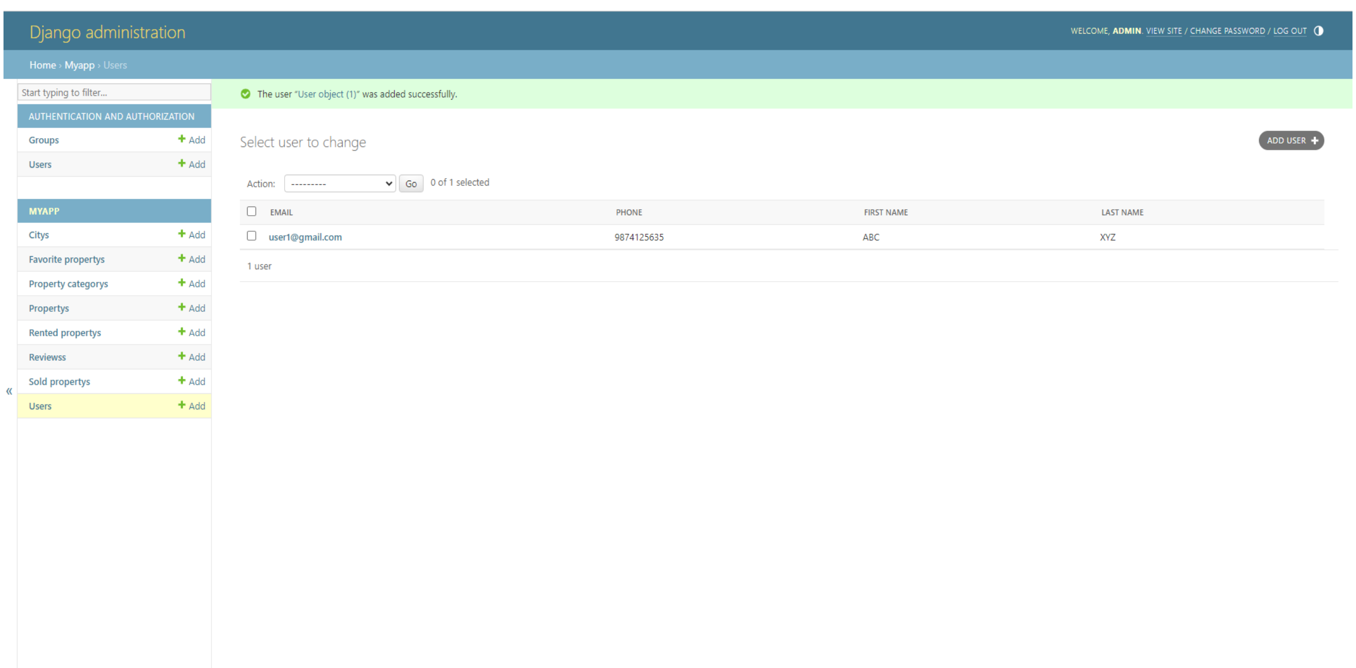
PROGRAM :

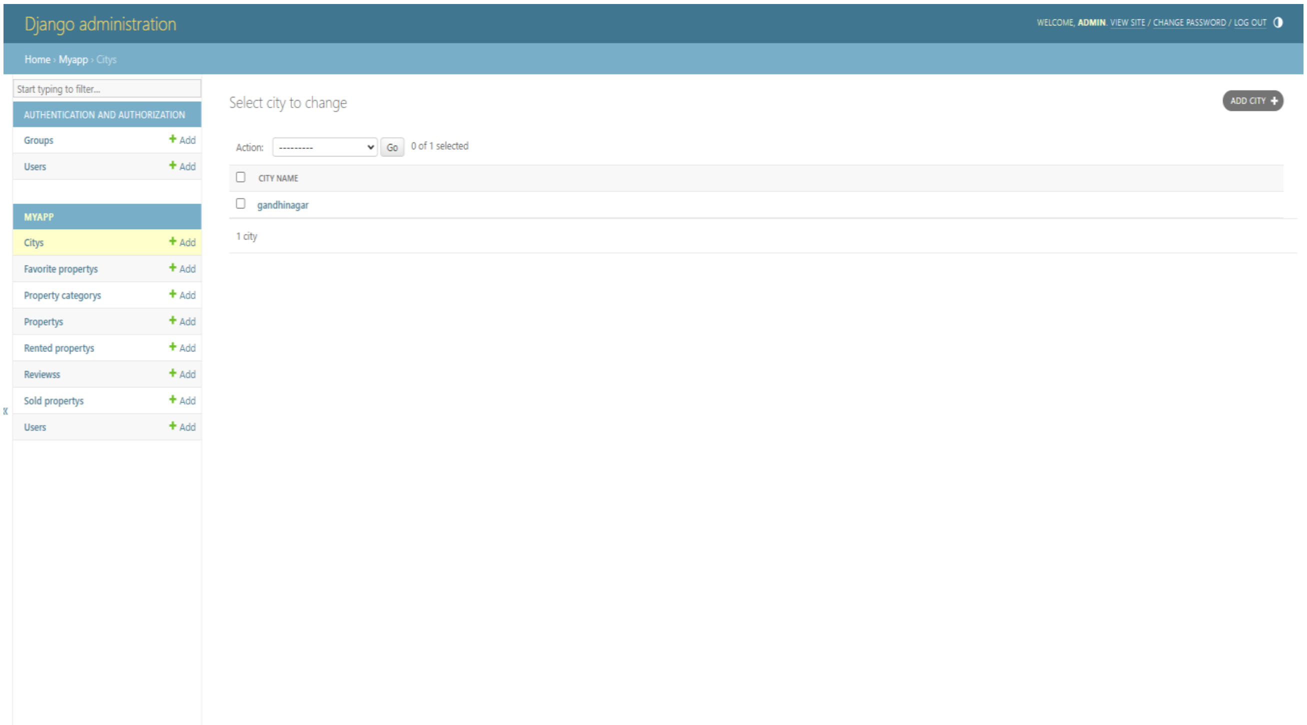
```
from django.db import models

7 usages
class User(models.Model):
    Email = models.EmailField()
    Password = models.CharField(max_length=25)  # You might want to consider using a more secure way to store passwords.
    Phone = models.CharField(max_length=15)
    First_Name = models.CharField(max_length=25)
    Last_Name = models.CharField(max_length=25)

3 usages
class PropertyCategory(models.Model):
    CATEGORY_NAME = models.CharField(max_length=25)

3 usages
class City(models.Model):
    CITY_NAME = models.CharField(max_length=25)

6 usages
class Property(models.Model):
    PROPERTY_NAME = models.CharField(max_length=25)
    PROPERTY_DESCRIPTION = models.TextField()
    PROPERTY_CATEGORY = models.ForeignKey(PropertyCategory, on_delete=models.CASCADE)
    LOCATION = models.CharField(max_length=25)
    SELLER = models.ForeignKey(User, on_delete=models.CASCADE, related_name='properties_for_sale')
    PRICE = models.IntegerField()
    BEDROOMS = models.IntegerField()
    CITYID = models.ForeignKey(City, on_delete=models.CASCADE)

2 usages
class RentedProperty(models.Model):
    PROPERTY = models.ForeignKey(Property, on_delete=models.CASCADE)
    USER = models.ForeignKey(User, on_delete=models.CASCADE)
    START_DATE = models.DateField()
    END_DATE = models.DateField()
    TRANSACTION_DATE = models.DateField()
    AMOUNT = models.IntegerField()
    STATUS_CHOICES = (
```

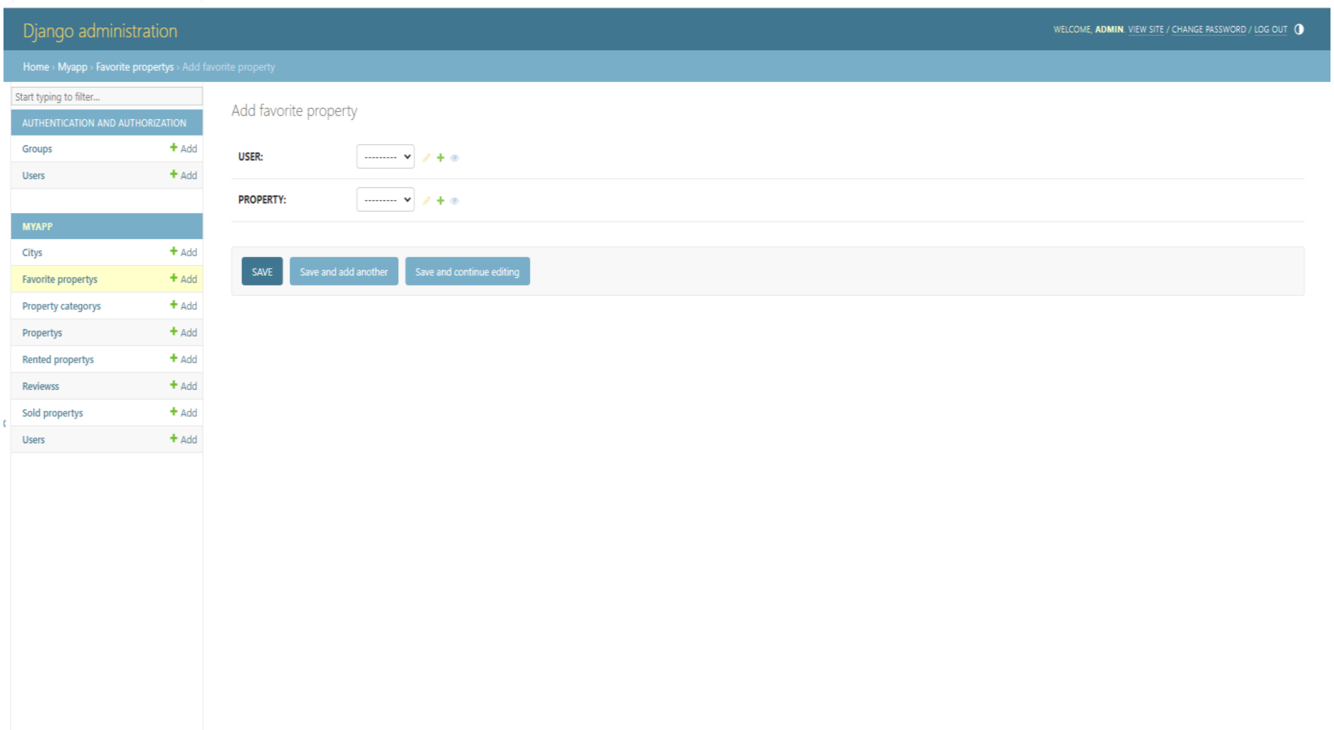
```python
from django.contrib import admin
from .models import User, PropertyCategory, City, Property, RentedProperty, SoldProperty, Reviews, FavoriteProperty

# Define custom admin classes for each model
1 usage
class UserAdmin(admin.ModelAdmin):
    list_display = ["Email", "Phone", "First_Name", "Last_Name"]

1 usage
class PropertyCategoryAdmin(admin.ModelAdmin):
    list_display = ["CATEGORY_NAME"]

1 usage
class CityAdmin(admin.ModelAdmin):
    list_display = ["CITY_NAME"]

1 usage
class PropertyAdmin(admin.ModelAdmin):
    list_display = ["PROPERTY_NAME", "PROPERTY_CATEGORY", "LOCATION", "SELLER", "PRICE", "BEDROOMS", "CITYID"]

1 usage
class RentedPropertyAdmin(admin.ModelAdmin):
    list_display = ["PROPERTY", "USER", "START_DATE", "END_DATE", "TRANSACTION_DATE", "AMOUNT", "STATUS"]

1 usage
class SoldPropertyAdmin(admin.ModelAdmin):
    list_display = ["PROPERTY", "USER", "TRANSACTION_DATE", "AMOUNT"]

1 usage
class ReviewsAdmin(admin.ModelAdmin):
    list_display = ["PROPERTY", "USER", "RATING", "COMMENT", "DATE_POSTED"]

1 usage
class FavoritePropertyAdmin(admin.ModelAdmin):
    list_display = ["USER", "PROPERTY"]

# Register the models with their respective custom admin classes
admin.site.register(User, UserAdmin)
```

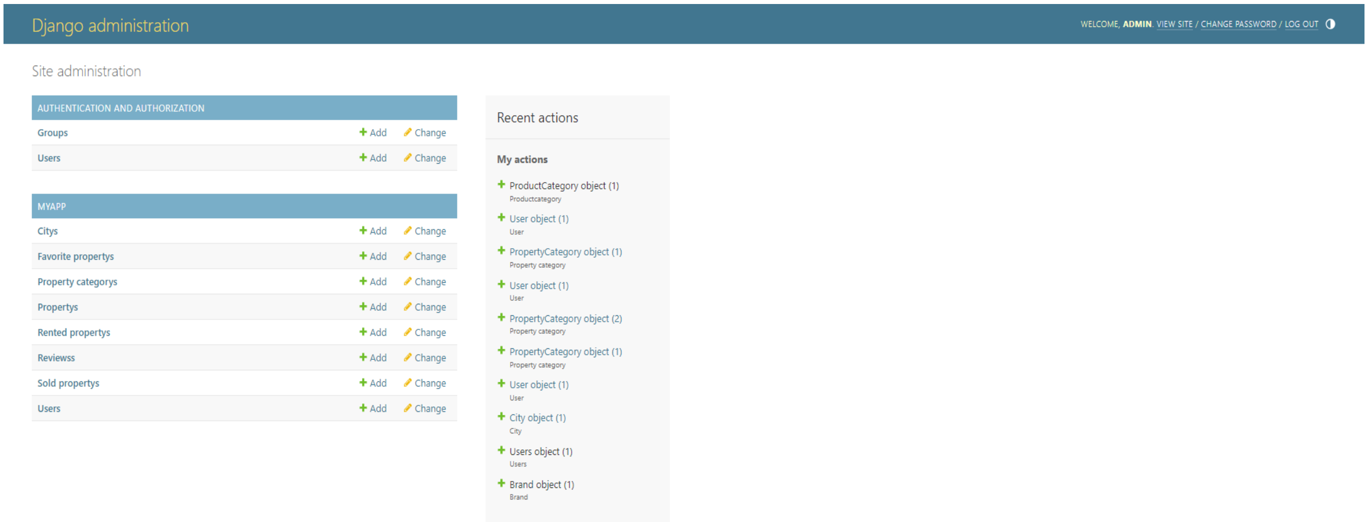
OUTPUT:


Django administration
WELCOME, ADMIN. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Myapp › Rented propertys › Add rented property
Start typing to filter...
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
MYAPP
Citys + Add
Favorite propertys + Add
Property categorys + Add
Propertys + Add
Rented propertys + Add
Reviewss + Add
Sold propertys + Add
Users + Add
Add rented property
PROPERTY:
USER:
START DATE: Today |
Note: You are 5.5 hours ahead of server time.
END DATE: Today |
Note: You are 5.5 hours ahead of server time.
TRANSACTION DATE: Today |
Note: You are 5.5 hours ahead of server time.
AMOUNT:
STATUS:
SAVE
Save and add another
Save and continue editing


Django administration
WELCOME, ADMIN. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Myapp › Users
Start typing to filter...
The user "User object (1)" was added successfully.
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
MYAPP
Citys + Add
Favorite propertys + Add
Property categorys + Add
Propertys + Add
Rented propertys + Add
Reviewss + Add
Sold propertys + Add
Users + Add
Select user to change
ADD USER +
Action: Go 0 of 1 selected
EMAIL
PHONE
FIRST NAME
LAST NAME
user1@gmail.com
9874125635
ABC
XYZ
1 user


Django administration
WELCOME, ADMIN. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Myapp › Citys
Start typing to filter...
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
MYAPP
Citys + Add
Favorite propertys + Add
Property categorys + Add
Propertys + Add
Rented propertys + Add
Reviewss + Add
Sold propertys + Add
Users + Add
Select city to change
ADD CITY +
Action: --------- Go 0 of 1 selected
CITY NAME
gandhinagar
1 city


Django administration
WELCOME, ADMIN. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Myapp › Favorite propertys › Add favorite property
Start typing to filter...
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
MYAPP
Citys + Add
Favorite propertys + Add
Property categorys + Add
Propertys + Add
Rented propertys + Add
Reviewss + Add
Sold propertys + Add
Users + Add
Add favorite property
USER: ---------
PROPERTY: ---------
SAVE
Save and add another
Save and continue editing


Django administration
WELCOME, ADMIN. VIEW SITE / CHANGE PASSWORD / LOG OUT
Site administration
AUTHENTICATION AND AUTHORIZATION
Groups Add Change
Users Add Change
MYAPP
Citys Add Change
Favorite propertys Add Change
Property categorys Add Change
Propertys Add Change
Rented propertys Add Change
Reviewss Add Change
Sold propertys Add Change
Users Add Change
Recent actions
My actions
ProductCategory object (1)
Productcategory
User object (1)
User
PropertyCategory object (1)
Property category
User object (1)
User
PropertyCategory object (2)
Property category
PropertyCategory object (1)
Property category
User object (1)
User
City object (1)
City
Users object (1)
Users
Brand object (1)
Brand